

HiDiTingV100 图形驱动

开发指南

文档版本 00B02

发布日期 2025-07-30

前言

概述

本文档主要介绍图形驱动开发相关内容，包括工作原理、功能描述、接口使用方法及注意事项。

产品版本

与本文档相对应的产品版本如下：

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B02	2025-07-30	在“3.2 功能描述”中更新硬件约束：。
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....	i
1 概述.....	1
1.1 图形显示系统介绍	1
1.2 图形显示系统架构图.....	2
1.3 API 接口文件目录	2
2 VAU	3
2.1 概述.....	3
2.2 功能描述.....	3
2.3 开发指引.....	5
3 DPU	8
3.1 概述.....	8
3.2 功能描述.....	8
3.3 开发指引.....	10
4 JPEG	12
4.1 概述.....	12
4.2 功能描述.....	13
4.3 开发指引.....	13
4.3.1 获取缩略图.....	13
4.3.2 解码后数据输出 yuv420 semi-planer	15
4.4 注意事项.....	16
5 LCD	17
5.1 概述.....	17
5.2 功能描述.....	17
5.3 开发指引.....	20

插图目录

图 1-1 图形显示系统架构 2

图 4-1 JPEG 解码微架构图..... 13

图 5-1 初始化配置流程图..... 19

表格目录

表 2-1 VAU 模块接口 3

表 3-1 DPU 模块接口 9

表 5-1 LCD 提供的接口 17

1 概述

1.1 图形显示系统介绍

1.2 图形显示系统架构图

1.3 API 接口文件目录

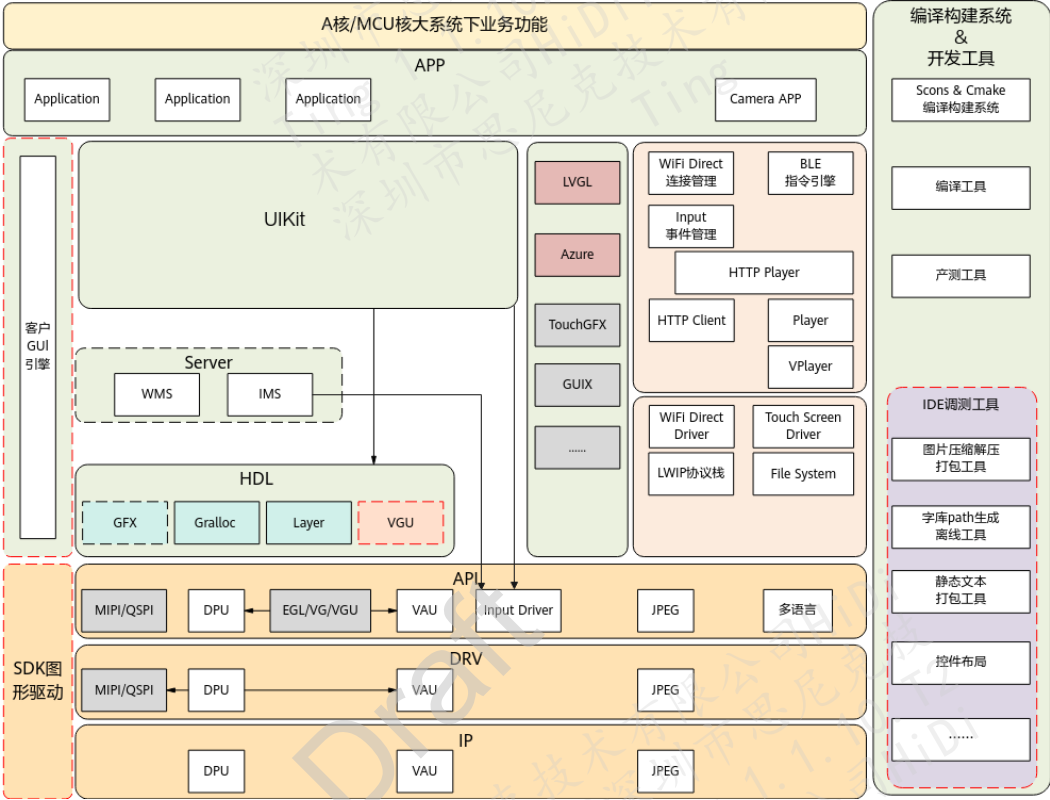
1.1 图形显示系统介绍

图形显示系统将绘制完成的内容在显示屏进行显示，主要包括 GUI 图形引擎、图形驱动、显示接口、显示设备，其中：

- GUI 图形引擎：包含窗口管理、布局、动画、输入事件、图形渲染及图形送显。详细信息参考《HiDiTingV100 UIKit 开发指南》。
- 图形驱动：包含 VAU、DPU、JPEG。详细信息参考后续章节。
- 显示接口：包含 MIPI、QSPI。
- 显示设备：智慧手表屏幕。

1.2 图形显示系统架构图

图1-1 图形显示系统架构



1.3 API 接口文件目录

头文件包含目录：./middleware/utils/common_headers/drivers/gpu

2 VAU

- 2.1 概述
- 2.2 功能描述
- 2.3 开发指引

2.1 概述

矢量加速单元（Vector Acceleration Unit）主要包含矢量绘制、矩阵变换、2.5D、多层混合叠加、低功耗、高性能（60@fps）、低内存等功能。

2.2 功能描述

VAU 模块提供如表 2-1 所示接口。

表2-1 VAU 模块接口

接口	描述
uapi_vau_create	返回对应任务的句柄。
uapi_vau_cancel	取消对应任务。
uapi_vau_destroy	销毁对应任务
uapi_vau_fill	快速填充颜色，对 surface 填充的颜色为 color 值，color 的格式支持：ARGB8888、RGB888。
uapi_vau_render	矢量图形绘制，主要用于绘制矢量基本图形。

接口	描述
uapi_vau_compose	多图层叠加，可支持叠加、仿射变换、ColorKey、ROP 等操作（多个操作一次性提交）。
uapi_vau_submit	提交任务。
uapi_vau_parse_cm p_header	解析压缩文件头。
uapi_vau_wait_for_ done	等待任务完成。

- 矩形填充

用某种颜色填充表面的某一矩形区域，矩形填充支持 Alpha 混和。

- 数据搬移

数据搬移过程中可以完成像素格式转换、Alpha 混和、ColorKey、ROP 等，也可进行缩放、旋转、镜像变换。

- 像素格式转换

Blit 模块支持以下像素格式转换：

- 源位图像素格式：ARGB8888、RGB888、RGB565、ARGB1555、ARGB4444、A1、A8。
- 目标位图像素格式：ARGB8888、RGB888、RGB565、ARGB1555、ARGB4444。

- Alpha 混和

将多个位图的像素值按照 Alpha 值进行加权求和，得到一个 Alpha 混和后的位图。Alpha 可来源于源位图、目标位图和全局 Alpha。

- ColorKey

- 对源位图进行 ColorKey，则把位图从源地址搬移到输出地址，在此过程中，颜色为关键色的位图像素使用目标位图数据替换。
- 对目标位图进行 ColorKey，则把位图从目标地址搬移到输出地址，在此过程中，颜色为与关键色不相同的位图像素用源位图数据替换。

- ROP

对源位图和目标位图的颜色值进行布尔运算。

2.3 开发指引

以使用 VAU 两层叠加为例，surface1 作为背景通过 VAU 纯色填充，surface2 作为前景，surface1 与 surface2 进行叠加。

步骤 1 设置 surface1 和 surface2 的属性配置：

```
#define TEST_VAU_W 454
#define TEST_VAU_H 454
#define TEST_VAU_PSRAM_OFFSET      0x6C200000
#define TEST_VAU_MAX_BUF_SIZE      0x10000000

ext_vau_surface src_surface[2] = {0};
ext_vau_surface dst_surface = {0};
ext_vau_surface_list surface_list = {0};
ext_vau_blit_opt opt = {0};

/* src_surface[0] background fill color red */
src_surface[0].width      = TEST_VAU_W;
src_surface[0].height     = TEST_VAU_H;
src_surface[0].phy_addr   = TEST_VAU_PSRAM_OFFSET + 0xD0000;
src_surface[0].color_fmt  = EXT_GFX_FMT_ARGB8888;
src_surface[0].compress_mode = EXT_GFX_COMPRESS_NONE;
src_surface[0].argb_order = EXT_GFX_FMT_ORDER_ARGB;
src_surface[0].rect.x     = 100;
src_surface[0].rect.y     = 100;
src_surface[0].rect.width = 256; /* 256:index */
src_surface[0].rect.height = 256; /* 256:index */
src_surface[0].stride     = 1816; /* 454 * 32 / 8 = 1816 */
src_surface[0].is_color_surface = TD_TRUE;
src_surface[0].color      = 0xffff0000;
memset_s((void *)src_surface[0].phy_addr, TEST_VAU_MAX_BUF_SIZE,
0xff, src_surface[0].stride * src_surface[0].height)

/* src_surface[1] foreground white */
src_surface[1].width      = 256; /* 256:index */
src_surface[1].height     = 256; /* 256:index */
src_surface[1].phy_addr   = TEST_VAU_PSRAM_OFFSET;
src_surface[1].color_fmt  = EXT_GFX_FMT_RGB565;
src_surface[1].argb_order = EXT_GFX_FMT_ORDER_ARGB;
src_surface[1].compress_mode = EXT_GFX_COMPRESS_NONE;
src_surface[1].rect.x     = 0;
src_surface[1].rect.y     = 0;
```

```

src_surface[1].rect.width    = src_surface[1].width;
src_surface[1].rect.height   = src_surface[1].height;
src_surface[1].stride        = 912; /* 454 * 16 / 8 = 908, 8字节对齐得912 */

memset_s((void *)src_surface[1].phy_addr, TEST_VAU_MAX_BUF_SIZE,
0x0, src_surface[1].stride * src_surface[1].height);
src_surface[1].alpha_info.global_alpha_en = TD_TRUE;
src_surface[1].alpha_info.global_alpha    = 0x50;
src_surface[1].alpha_info.pixel_alpha_en  = TD_TRUE;
src_surface[1].alpha_info.premult_en      = TD_TRUE;

```

步骤 2 设置 dst surface 的属性:

```

/* dst surface */
dst_surface.width          = TEST_VAU_W;
dst_surface.height         = TEST_VAU_H;
dst_surface.phy_addr       = TEST_VAU_PSRAM_OFFSET + 0xD0000;
dst_surface.color_fmt       = EXT_GFX_FMT_ARGB8888;
dst_surface.compress_mode   = EXT_GFX_COMPRESS_NONE;
dst_surface.rgb_order       = EXT_GFX_FMT_ORDER_ARGB;
dst_surface.rect.x          = src_surface[0].rect.x;
dst_surface.rect.y          = src_surface[0].rect.y;
dst_surface.rect.width      = src_surface[0].rect.width;
dst_surface.rect.height     = src_surface[0].rect.height;
dst_surface.stride          = 1816; /* 454 * 32 / 8 = 1816 */
opt.blend_opt.blend_enable  = TD_TRUE;
opt.blend_opt.blend_cmd     = EXT_VAU_BLEND_CMD_SRCOVER; /* 正常绘制显示, 上下层绘制叠加 */

opt.blend_opt.out_alpha_mode = EXT_VAU_OUT_ALPHA_FROM_FOREGROUND; /* alpha来自前景 */

surface_list.src_surface_cnt = 2; /* 2:index */
surface_list.src_surface     = src_surface;
surface_list.dst_surface     = &dst_surface;
surface_list.opt              = &opt;

```

步骤 3 执行一次 blit 操作:

```

td_s32 ret;
td_handle handle;
handle = uapi_vau_create();
if (handle <= 0) {
    printf("uapi_vau_create failed\n");
}

```

```
        return EXT_FAILURE;
    }
    ret = uapi_vau_compose(handle, surface_list);
    if (ret != EXT_SUCCESS) {
        printf("uapi_vau_compose failed\n");
        return ret;
    }
    ret = uapi_vau_submit(handle, TD_TRUE, 1000); /* 1000 ms timeout */
    if (ret != EXT_SUCCESS) {
        uapi_vau_cancel(handle);
        printf("uapi_vau_submit failed\n");
        return ret;
    }
    uapi_vau_destroy(handle);
```

----结束

3 DPU

3.1 概述

3.2 功能描述

3.3 开发指引

3.1 概述

显示处理单元 DPU (Display Processing Unit) 主要对图形显示通路的处理, 包含多层叠加、自研压缩数据解压、局部刷新、低功耗等功能。

3.2 功能描述

主要功能:

- 支持两层在线叠加。
- 支持 RTOS 操作系统。
- 支持 SRAM 存储访问。
- 支持 PSRAM 存储访问。
- 支持低功耗。
- 支持输入分辨率可设。
- 支持 CMD 模式刷新显示。
- 支持对接 MIPI 接口实现图形显示。
- 支持 DFX。

- 支持低 Flash 下 DPU 运行。
- 支持低内存下 DPU 运行。
- 支持图层 ColorKey。

硬件约束：

- 申请地址 16Byte 对齐。
- 输入 stride 要求 16Byte 对齐。
- G0/G1 支持的颜色格式：RGB888/RGB565。

DPU 模块提供以下接口：

表3-1 DPU 模块接口

接口	说明
uapi_dpu_create_layer	创建图层。
uapi_dpu_destroy_layer	销毁图层。
uapi_dpu_set_layer_alpha	设置图层全局 Alpha。
uapi_dpu_get_layer_alpha	获取图层全局 Alpha。
uapi_dpu_set_layer_pos	设置图层在屏上的显示位置。
uapi_dpu_get_layer_pos	获取图层在屏上的显示位置。
uapi_dpu_set_layer_zorder	设置图层 Z 序。
uapi_dpu_get_layer_zorder	获取图层 Z 序。
uapi_dpu_show_layer	显示图层。
uapi_dpu_hide_layer	隐藏图层。
uapi_dpu_set_layer_info	设置图层信息。
uapi_dpu_get_layer_info	获取图层信息。
uapi_dpu_set_layer_colorkey	设置图层的 ColorKey。
uapi_dpu_get_layer_colorkey	获取图层的 ColorKey。
uapi_dpu_refresh_layers	刷新显示。

接口	说明
uapi_dpu_wait_layer_vsync	等待显示完成。
uapi_dpu_set_layer_vblank	设置等待标记。
uapi_dpu_get_layer_vblank	获取等待标记。
uapi_dpu_query_layer_state	查询图层的显示完成状态。

3.3 开发指引

步骤 1 通过接口 `uapi_dpu_create_layer` 创建显示图层，并返回图层句柄：

```
td_s32 ret;
td_handle dpu_handle;
dpu_surface dpu_canvas = { 0 };
dpu_layer_info layer_info = { 0 };
/* dst_surface vau的输出surface */
layer_info.width          = dst_surface->width;
layer_info.height         = dst_surface->height;
layer_info.layer_id       = EXT_DPU_LAYER_ID1;
layer_info.layer_fmt      = dst_surface->color_fmt;
layer_info.compress_mode  = dst_surface->compress_mode;
/* NORMAL模式（双canvas送显模式） */
layer_info.buf_mode       = EXT_DPU_BUFFER_MODE_NORMAL;
ret = uapi_dpu_create_layer(&layer_info, &dpu_handle);
if (ret != EXT_SUCCESS) {
    printf("call uapi_dpu_create_layer fail, ret:0x%x\n", ret);
    return ret;
}
```

步骤 2 通过接口 `uapi_dpu_set_layer_info` 设置图层显示信息：

```
dpu_canvas.width          = dst_surface->width;
dpu_canvas.height         = dst_surface->height;
dpu_canvas.stride         = dst_surface->stride;
dpu_canvas.size           = dpu_canvas.height * dpu_canvas.stride;
dpu_canvas.phy_addr       = dst_surface->phy_addr;
dpu_canvas.vir_addr       = dst_surface->vir_addr;
dpu_canvas.update_rect    = TD_NULL;
```

```
dpu_canvas.color_fmt      = dst_surface->color_fmt;
dpu_canvas.compress_mode  = dst_surface->compress_mode;
ret = uapi_dpu_set_layer_info(dpu_handle, &dpu_canvas);
if (ret != EXT_SUCCESS) {
    printf("call uapi_dpu_set_layer_info fail, ret:0x%x\n", ret);
    return ret;
}
```

步骤 3 调用接口 uapi_dpu_refresh_layers 刷新显示内容:

```
ret = uapi_dpu_refresh_layers();
if (ret != EXT_SUCCESS) {
    printf("call uapi_dpu_refresh_layers fail, ret:0x%x\n", ret);
    return ret;
}
```

----结束

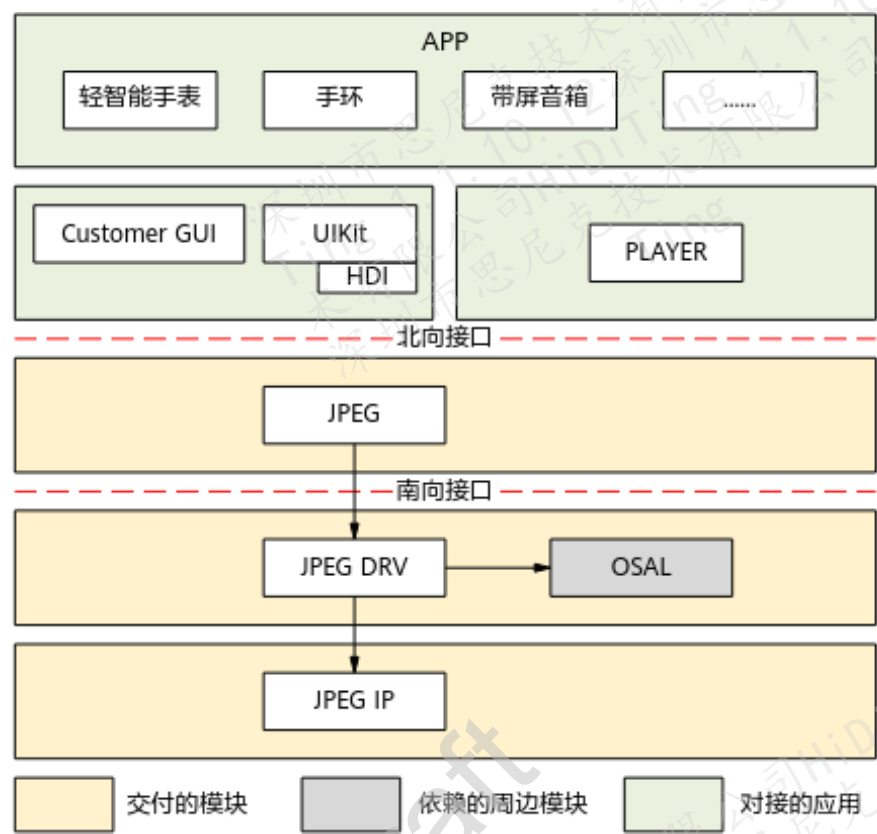
4 JPEG

- 4.1 概述
- 4.2 功能描述
- 4.3 开发指引
- 4.4 注意事项

4.1 概述

穿戴手表对体验的需求越来越高，如视频预览（手表控制手机拍照，在手表端可看到要拍照的内容）、本地视频播放（如视频表盘），而这些应用需要支持 JPEG 解码的功能。

图4-1 JPEG 解码微架构图



4.2 功能描述

- 支持文件方式输入。
- 支持内存方式输入。
- 支持解码输出 yuv420 semi-planer 格式的数据。
- 支持解码缩小 1 倍、2 倍、4 倍、8 倍。
- 支持获取缩略图。

4.3 开发指引

JPEG 提供的接口说明请参考 API 说明，后续章节通过 demo 来指导应用如何使用。

4.3.1 获取缩略图

```
void demo_get_exif_data(char *file_name)
```

```
{
    int ret;
    static int cnt = 0;
    FILE *in_file = NULL;
    FILE *ou_file = NULL;
    struct jpeg_decompress_struct dinfo = { 0 };
    struct jpeg_source_struct src = { 0 };

    in_file = fopen(file_name, "rb");
    if (in_file == NULL) {
        return;
    }

    /* 设置输入流信息 */

    src.infile = in_file;

    /* 缩略图操作使能 */

    dinfo.thumbnail_en = TD_TRUE;

    /* 创建解码器, 该函数包含对文件头的解析处理 */

    ret = uapi_jpeg_decompress_create(&dinfo, &src, TD_FALSE);
    if (ret != 0) {
        fclose(in_file);
        return;
    }

    /* 缩略图要写入的文件路径和文件名 */

    snprintf(file_name, SAMPLE_JPEG_EXIF_FILE_PATH_LEN, "%s/out_%d.jpg",
SAMPLE_JPEG_EXIF_FILE_DIR, cnt++);

    /* 将缩略图数据写到文件中 */

    ou_file = fopen(file_name, "wb");
    fwrite(dinfo.thumbnail.buffer, 1, dinfo.thumbnail.size, ou_file);
    fclose(ou_file);

    /* 销毁解码器 */

    uapi_jpeg_decompress_destroy(&dinfo);

    fclose(in_file);
    return;
}
```

4.3.2 解码后数据输出 yuv420 semi-planer

```
void demo_decompress_to_yuvsp(char *file_name)
{
    int ret;
    FILE *in_file = NULL;
    struct jpeg_decompress_struct dinfo = { 0 };
    struct jpeg_source_struct src = { 0 };
    struct jpeg_dst_struct dst = { 0 };

    in_file = fopen(file_name, "rb");
    if (in_file == NULL) {
        return;
    }

    /* 设置输入流信息 */
    src.infile = in_file;

    /* 缩小1倍 */
    dinfo.scale = JPEG_SCALE_1;

    /* 解码输出格式和图片格式保持一致 */
    dinfo.output_yuv420sp = TD_TRUE;

    /* 创建解码器 */
    ret = uapi_jpeg_decompress_create(&dinfo, &src, TD_FALSE);
    if (ret != 0) {
        fclose(in_file);
        return;
    }

    /* 根据图片信息分配内存 */
    dst.stride = dinfo.output.stride[0];
    dst.uv_stride = dinfo.output.stride[1];
    dst.size = dinfo.output.size[0] + dinfo.output.size[1];
    dst.uv_offset = dinfo.output.size[0];

    /* 内存分配 */
    dst.phy_buf = mpi_mmz_new(dst.size, 64, NULL, "JPEG");
    if (dst.phy_buf == 0) {
        fclose(in_file);
        return;
    }
}
```

```
}

dst.vir_buf = (td_char *)mpi_mmz_map(dst.phy_buf, 0);
if (dst.vir_buf == NULL) {
    mpi_mmz_delete(dst.phy_buf);
    fclose(in_file);
    return;
}

/* 启动解码 */
uapi_jpeg_decompress_start(&dinfo, &dst);

/* 销毁解码器 */
uapi_jpeg_decompress_destroy(&dinfo);

/* 释放资源 */
mpi_mmz_unmap(dst.phy_buf);
mpi_mmz_delete(dst.phy_buf);

fclose(in_file);
return;
}
```

4.4 注意事项

只支持 baseline，数据精度为 8 的 JPEG 图片。

5 LCD

5.1 概述

5.2 功能描述

5.3 开发指引

5.1 概述

LCD 为屏端控制驱动，提供显示、进出睡眠、进出 IDLE、读写屏端控制寄存器、设置获取屏幕亮度等功能。

5.2 功能描述

LCD 模块提供以下接口：

表5-1 LCD 提供的接口

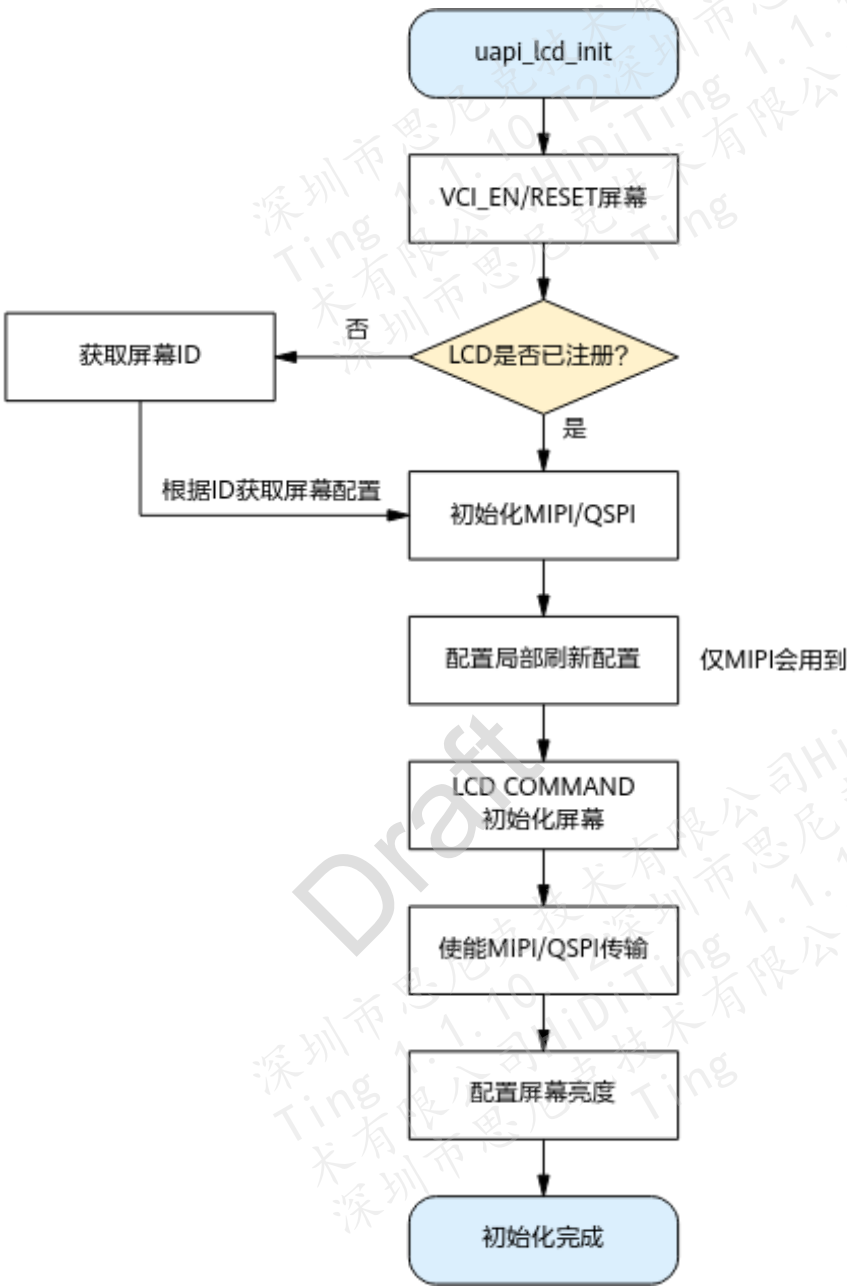
接口	描述
uapi_lcd_info_register	屏幕信息注册，包括屏幕分辨率、帧率、通信串口类型及指令集等。
uapi_lcd_init	LCD 驱动初始化。
uapi_lcd_deinit	LCD 驱动去初始化。
uapi_lcd_get_status	获取屏幕当前状态。
uapi_lcd_enter_sleep_mode	使屏端进入睡眠模式。

接口	描述
uapi_lcd_exit_sleep_mode	使屏端退出睡眠模式。
uapi_lcd_enter_idle_mode	使屏端进入 IDLE 模式。
uapi_lcd_exit_idle_mode	使屏端退出 IDLE 模式。
uapi_lcd_te_on	使能 te 信号。
uapi_lcd_te_off	关闭 te 信号。
uapi_lcd_set_brightness	设置屏幕亮度。
uapi_lcd_get_brightness	获取当前屏幕亮度。
uapi_lcd_write_reg	将数据写入屏端寄存器。
uapi_lcd_read_reg	从屏端寄存器读取数据。

初始化

uapi_lcd_init 接口支持屏端复位、串口初始化、屏端初始化、送显使能等配置，主要配置流程如图 5-1 所示。

图5-1 初始化配置流程图



通信串口说明

驱动内部支持两种串口协议：QSPI 协议与 MIPI 协议。完成硬件环境搭建后，通过 `uapi_lcd_info_register` 接口注册屏幕信息，屏蔽了底层协议的差异。

5.3 开发指引

步骤 1 LCD 信息注册:

信息注册的参数结构:

```
typedef struct {  
    lcd_bus_type bus_type;  
    combo_dev_cfg_t *bus_cfg;  
    uint16_t x_start;  
    uint16_t y_start;  
    uint32_t fps;  
    uint32_t lcd_id;  
    uint32_t ddic_id;  
    lcd_cmds_sequ *display_on_cmds;  
    lcd_cmds_sequ *display_off_cmds;  
    lcd_cmds_sequ *enter_idle_cmds;  
    lcd_cmds_sequ *exit_idle_cmds;  
    lcd_cmds_sequ *te_on_cmds;  
    lcd_cmds_sequ *te_off_cmds;  
    lcd_cmds_sequ *set_brightnes_cmds;  
    lcd_cmds_sequ *sleep_in_cmds;  
    lcd_cmds_sequ *sleep_out_cmds;  
    ext_lcd_read_info lcd_id_info;  
} lcd_drv_cfg;
```

combo_dev_cfg_t 结构体定义:

```
typedef struct {  
    uint32_t devno;  
    short lane_id[LANE_MAX_NUM];  
    out_mode_t out_mode;  
    video_mode_t video_mode;  
    out_format_t out_format;  
    sync_info_t sync_info;  
    pck_en_t pck_en;  
    uint32_t phy_data_rate;  
    uint32_t pixel_clk;  
} combo_dev_cfg_t;
```

sync_info_t 结构体定义:

```
typedef struct {  
    uint16_t hsa_pixels;
```

```

uint16_t      hbp_pixels;
uint16_t      hact_pixels;
uint16_t      hfp_pixels;
uint16_t      vsa_lines;
uint16_t      vbp_lines;
uint16_t      vact_lines;
uint16_t      vfp_lines;
} sync_info_t;

```

指令集结构体定义:

```

typedef struct lcd_cmd_sequ {

    uint8_t dcs_flag; /* 当前数据是否通过DCS协议命令下发，为0表示不是DCS命令 */

    uint8_t data_len; /* payload length, 小于等于64 */

    uint8_t data[MAX_DATA_LENGTH]; /* data内包含cmd和参数，有效数据最短为1（例如0x11），
    最长为128 */

    uint16_t delay_ms; /* 发送命令后等待时间，单位：ms */
} lcd_cmd_sequ;
typedef struct lcd_cmds_sequ {

    lcd_cmd_sequ *cmd; /* dsi命令数组 */

    uint16_t cmd_cnt; /* dsi命令个数 */
} lcd_cmds_sequ;

```

LCD 信息注册示例代码:

BUS配置信息

```

/* MIPI配置参数,根据实际屏幕信息修改 */
combo_dev_cfg_t g_mipi_config = {

    .devno = 0,

    .lane_id = {0, -1},

    .out_mode = OUT_MODE_DSI_CMD,

    .out_format = OUT_FORMAT_RGB_24BIT,

    .sync_info = {

        .hsa_pixels = 10,    // hsa 10 pixels

        .hbp_pixels = 10,    // hbp 10 pixels

        .hact_pixels = 454,  // hact 454 pixels

        .hfp_pixels = 10,    // hfp 10 pixels

        .vsa_lines = 4,      // vsa 4 lines

        .vbp_lines = 12,     // vbp 12 lines

        .vact_lines = 454,   // vact 454 lines
    }
}

```

```

        .vfp_lines    = 10,    // vfp 10 lines
    },
    .phy_data_rate = 480,      // 480Mbps
    .pixel_clk = 20000,       // 20000KHz
};
/* QSPI屏幕只需要传入分辨率及像素格式, 定义如下 */
combo_dev_cfg_t g_qspi_config = {
    .out_format = OUT_FORMAT_RGB_24BIT,
    .sync_info = {
        .hact_pixels    = 194,    // hact 368 pixels
        .vact_lines     = 368,    // vact 368 lines
    },
};

```

指令集

```

/* display off sequence for common lcd */
lcd_cmd_sequ g_common_cmd_display_off[] = {
    {1, 2, {0xfe, 0x00}, 0},    /* select default cmd page */
    {1, 1, {0x28}, 0},         /* display off */
    {1, 1, {0x10}, 120},       /* sleep in */
};
/* sleep in sequence for common lcd */

const lcd_cmd_sequ g_common_cmd_sleep_in[] = {
    {1, 2, {0xfe, 0x00}, 0},    /* select default cmd page */
    {1, 1, {0x28}, 0},         /* display off */
    {1, 1, {0x10}, 120},       /* sleep in */
};
/* sleep out sequence */

const lcd_cmd_sequ g_common_cmd_sleep_out[] = {
    {1, 2, {0xfe, 0x00}, 0},    /* select default cmd page */
    {1, 1, {0x11}, 120},       /* sleep out */
    {1, 1, {0x29}, 0},         /* display on */
};
/* idle in sequence */
lcd_cmd_sequ g_common_cmd_idle_in[] = {
    {1, 2, {0xfe, 0x00}, 0},
    {1, 1, {0x39}, 0},
};
/* idle out sequence */
lcd_cmd_sequ g_common_cmd_idle_out[] = {
    {1, 2, {0xfe, 0x00}, 0},

```

```

        {1, 1, {0x38}, 0},
    };
    /* te on sequence */
    lcd_cmd_sequ g_common_cmd_te_on[] = {
        {1, 2, {0xfe, 0x00}, 0},
        {1, 1, {0x35}, 0},
    };
    /* te off sequence */
    lcd_cmd_sequ g_common_cmd_te_off[] = {
        {1, 2, {0xfe, 0x00}, 0},
        {1, 1, {0x34}, 0},
    };
    /* set brightness sequence */
    lcd_cmd_sequ g_common_cmd_brightness[] = {
        {1, 2, {0xfe, 0x00}, 0},
        {1, 2, {0x51, 0xff}, 0},
    };
    /* init sequence, 根据屏端提供的手册输入 */
    lcd_cmd_sequ g_screen_init[] = {
        {1, 2, {0xfe, 0x01}, 0},
        {1, 2, {0x30, 0x43}, 0}, /* set LCD frequency to 15HZ */
        {1, 2, {0xfe, 0x00}, 0},
        {1, 2, {0x3a, 0x77}, 0},
        {1, 2, {0x53, 0x20}, 0},
        {1, 2, {0x51, 0xff}, 0},
        {0, 5, {0x2a, 0x00, 0x0e, 0x01, 0xd3}, 0},
        {0, 5, {0x2b, 0x00, 0x00, 0x01, 0xc5}, 0},
        {1, 2, {0x35, 0x00}, 0},
        {1, 1, {0x11}, 120},
        {1, 1, {0x29}, 0},
    };

```

信息注册

```

lcd_drv_cfg g_lcd_info = {
    .bus_type           = BUS_DISPLAY_MIPI, /* BUS_DISPLAY_QSPI */
    .bus_cfg            = &g_mipi_config, /* g_qspi_config */
    .x_start            = 0xe,
    .y_start            = 0,
    .fps                = 60,
    .lcd_id             = 0xf00,
    .display_on_cmds    = &g_screen_init,
    .display_off_cmds   = &g_display_off_cmds,
    .enter_idle_cmds    = &g_idle_in_cmds,
}

```

```

        .exit_idle_cmds      = &g_idle_out_cmds,
        .te_on_cmds         = &g_te_on_cmds,
        .te_off_cmds        = &g_te_off_cmds,
        .set_brightness_cmds = &g_brightness_cmds,
        .sleep_in_cmds      = &g_sleep_in_cmds,
        .sleep_out_cmds     = &g_sleep_out_cmds,
        .lcd_id_info.reg_addr = 0x04,
        .lcd_id_info.read_len = 3,
    };
    ext_errno ret = uapi_lcd_info_register(&g_lcd_info);
    if (ret != EXT_ERR_SUCCESS) {
        ...
    }

```

步骤 2 LCD 初始化:

```

ext_errno ret = uapi_lcd_init();
if (ret != EXT_ERR_SUCCESS) {
    ...
}

```

步骤 3 送显。参考 VAU 及 DPU 接口说明，本章不做说明。

步骤 4 设置亮度或其他控制功能，以设置亮度为例:

```

uint32_t brightness = 0x80;
ext_errno ret = uapi_lcd_set_brightness(brightness);
if (ret != EXT_ERR_SUCCESS) {
    ...
}

```

步骤 5 读写其他控制寄存器。

- 写操作:

```

lcd_cmd_sequ wdata = {0};
wdata.data[0] = 0x3A; /* 修改像素格式 */

wdata.data[1] = 0x77; /* 像素格式为RGB888 */
wdata.data_len = 2;
ext_errno ret = uapi_lcd_write_reg(&wdata, 1);
if (ret != EXT_ERR_SUCCESS) {
    ...
}

```

- 读操作:

```

uint8_t data;

```

```
ext_errno ret = uapi_lcd_read_reg(0xC, &data, 1); /* 0xC读像素格式 */  
if (ret != EXT_ERR_SUCCESS) {  
    ...  
}
```

----结束

Draft